

Big Data Analytics: Mini Project 3

Project Title: Real-Time Recommendation System

Assignment Weight	Team Size	Due Date
10 Points	2 Students	12th May 2026

1. Project Objective

The goal of this project is to design and implement an end-to-end big data system that combines batch machine learning and real-time streaming analytics using Apache Spark and Kafka.

Students will:

- Build a scalable recommendation system using ALS
- Process real-time data streams using Spark Structured Streaming
- Integrate machine learning models with streaming pipelines
- Analyze how modern systems handle historical + real-time data
- Evaluate system performance and real-time behavior

2. Problem Definition

Each team will build a real-time recommendation system that:

- Learns user preferences from historical data
- Processes live user interactions via streaming
- Generates dynamic recommendations in real time

Example Scenarios

- A user interacts with an item → recommendations update instantly
- An item suddenly becomes popular → system detects trending behavior
- High activity patterns → system triggers anomaly alerts

To ensure diversity and deeper understanding, each team must customize their system.

A. Domain Selection (Choose ONE)

- Movies (default – MovieLens dataset)
- Music recommendation
- E-commerce products
- Books
- Games
- Restaurants / food

B. System Focus (Choose ONE)

1. Personalization Focus

- Improve recommendation quality
- Address cold-start problem
- Apply user segmentation

2. Real-Time Intelligence Focus

- Detect trending items
- Capture rating spikes
- Emphasize streaming impact

3. System Optimization Focus

- Improve latency and performance
- Optimize partitioning strategy
- Analyze throughput

Each team must clearly document:

- Selected domain
- Selected focus
- How their solution differs from others
- Additional challenges introduced

Teams with identical implementations will lose marks in Discussion & Innovation.

3. Dataset Requirements

Requirements:

- Size: 500 K records or more
- Format: (user_id, movie_id, rating, timestamp)
- Must include sufficient users/items for collaborative filtering

Justification:

Students must explain:

- Why the dataset fits the selected domain
- Why distributed processing is needed
- Challenges in data

4. Machine Learning Requirements (Batch Processing)

Students must:

Data Preprocessing:

- Clean and validate ratings
- Handle missing or invalid values

Model Training:

- Use ALS algorithm from Spark MLlib
- Split dataset:
 - 80% Training
 - 20% Testing

Evaluation:

- Compute RMSE
- If RMSE > 1.5:
 - Tune at least one parameter:
 - rank
 - regParam
 - maxIter

5. Streaming Requirements (Kafka + Spark Streaming)

Kafka Producer:

- Python-based producer
- Generate streaming events:

```
{
  "user_id": 10,
  "item_id": 200,
  "rating": 4.0,
  "timestamp": "..."}
}
```

Kafka Topic:

- Minimum 2 partitions
- Teams must justify partitioning strategy

6. Streaming Processing & Analytics

Using Spark Structured Streaming:

Core Requirements

- Consume Kafka stream
- Parse JSON safely
- Handle malformed records

Window Analytics

- Window size: 30 seconds
- Slide: 10 seconds

Compute

- Average rating per item
- Number of interactions per user

Implement one custom streaming metric, such as:

- **Trending score**
- **Engagement score**
- **Rating variance**

7. ML + Streaming Integration

Students must integrate batch ML with streaming data:

Minimum Requirement

- Generate Top-5 recommendations per incoming user
- Combine:
 - Historical data
 - Streaming updates
- Adapt recommendations dynamically
- **(bonus - 1 point)** Recommendations must be generated with < 5 seconds latency

8. Alert System & Late Data Handling

Alert System:

Trigger alerts when:

- Item rating exceeds threshold (e.g., > 4.5)
- User activity spikes

Example

ALERT: Item 120 is trending

Late Data Handling (Mandatory)

- How late data is handled
- Whether it is dropped or processed

9. Deliverables

Each team must submit:

1. Project Report must include:

- System Architecture Diagram
(Kafka → Spark → ML → Output)
- Machine Learning Component
- Streaming Component
- Integration Explanation
- Results (recommendations + alerts)
- Latency measurement
- Custom metric explanation
- Challenges & lessons learned

2. Code Submission

- ALS training code
- Kafka producer
- Spark streaming application
- Clear run instructions

10. Evaluation Criteria

Criteria	Weight
Data Preprocessing & ML Model	15%
RMSE Evaluation & Tuning	10%
Kafka Setup & Partitioning Strategy	10%
Streaming Pipeline Correctness	10%
Window Analytics	10%
ML + Streaming Integration	10%
Alert System & Watermarking	5%
Documentation Quality	20%
Discussion	10%

11. Bonus: Dashboard Analytics (+2 Points)

Students can earn up to +2 bonus points by building a dashboard.

Requirements

- Real-time or near real-time visualization
- Include at least 3:
 - Recommendations
 - Trending items
 - User activity
 - Alerts
 - Streaming metrics